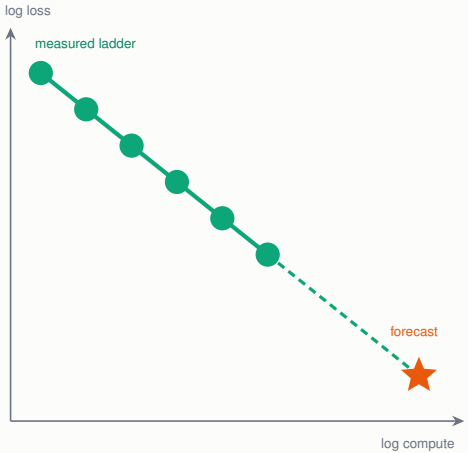
LECTURE NOTES · PRETRAINING

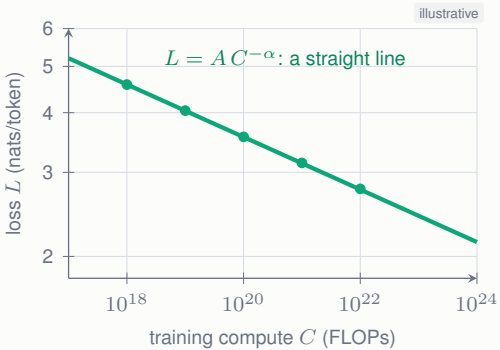
Scaling Laws, Visually

How loss falls with parameters, data and compute; the history before language models; the key results; and how to design, run and read a scaling experiment of your own.



What a scaling law is, and why anyone cares

An empirical regularity: a model family's loss falls as a power of the resource spent, smoothly, over many decades.



Log axes on both sides. Each decade of compute lowers loss by the same factor, $10^{-\alpha}$.

N parameters
how big the model is; sets the cost of every token, in training and at inference

D data
how many tokens it sees; sets the information available to fit

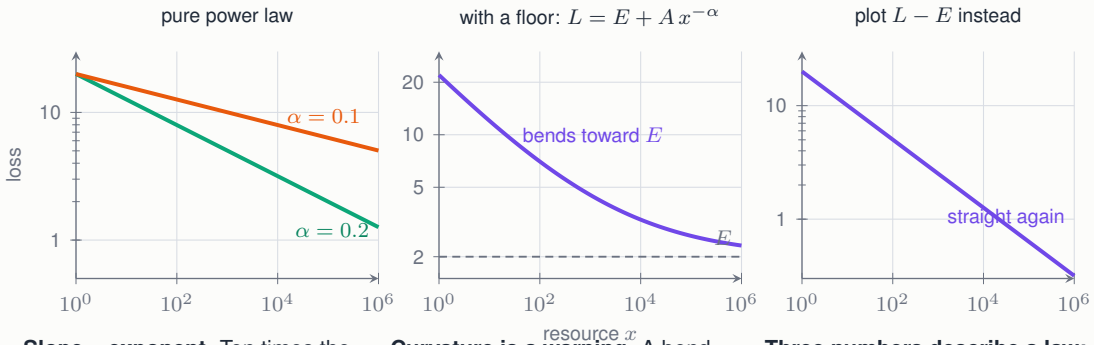
C compute
roughly $6\,N\,D$ FLOPs for a dense transformer; what you pay for

Why it matters

Forecast: predict the loss of a run before paying for it. **Budget:** split a fixed compute budget between N and D. **Compare:** measure a recipe change as compute saved, not as a single noisy number.

Reading a log-log plot

A pure power law is a line whose slope is the exponent. A floor bends it, and subtracting the floor straightens it again.



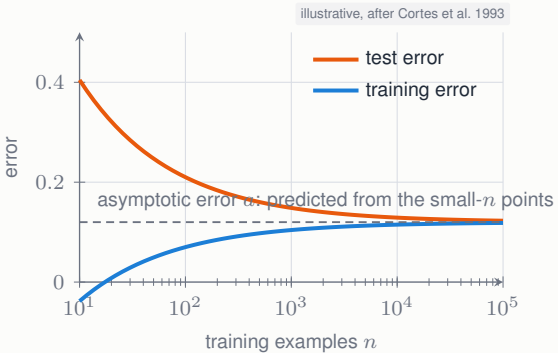
Slope = exponent. Ten times the resource multiplies the excess loss by $10^{-\alpha}$: for $\alpha = 0.05$ that is only 11%.

Curvature is a warning. A bend means a floor, a regime change, or a badly tuned point; do not fit a line through it.

Three numbers describe a law: the exponent α, the scale A and the floor E. Papers differ mostly in how they estimate them.

History I: learning curves, 1990s

Long before transformers, statisticians fit power laws to error versus training-set size and extrapolated them.



Both curves approach the same asymptote from opposite sides as $a \pm b\, n^{-\alpha}$; fitting the two together pins down a before the large run exists.

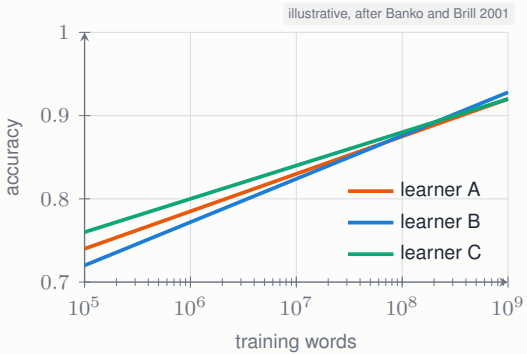
Cortes, Jackel, Solla, Vapnik, Denker (NIPS 1993). Fit test and training error against n with power laws and read off the asymptotic error, to choose between classifiers without training them to the end. The idea of a ladder is here already.

Amari (1992–93). Statistical mechanics of learning curves: generalization error falls like $1/n$ for well-specified models; the exponent is a property of the problem.

Vapnik. Uniform-convergence bounds with $\sqrt{h/n}$ rates: worst-case laws, which the empirical curves beat in practice.

History II: more data beats cleverer algorithms, 2001–2017

In NLP and then vision, accuracy kept rising log-linearly with data over three decades, for every learner tried.



The ranking of learners changes with data size, and none of the curves flattens within the range tested.

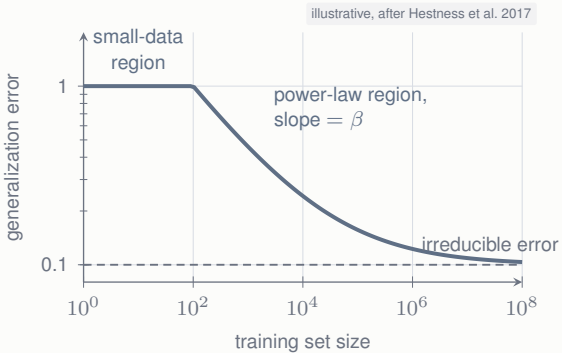
Banko and Brill (ACL 2001). Confusion-set disambiguation with corpora from one million to one billion words: accuracy rose roughly log-linearly for all learners, and the simplest ones caught up. Their conclusion: spend on data before spending on algorithms.

Halevy, Norvig, Pereira (2009). "The unreasonable effectiveness of data": simple models on web-scale corpora beat elaborate ones on small ones.

Sun et al. (2017). JFT-300M: vision accuracy still rising logarithmically with data at 300 million images; model capacity has to grow with it.

History III: deep learning scaling is predictable, 2017–2019

Hestness et al. measured learning curves across domains: the same three-region shape, with task-specific exponents.



The exponent β is set by the task and data; changing the architecture shifts the curve up or down but, in their data, leaves the slope alone.

Hestness et al. (Baidu, 2017). Machine translation, language modeling, speech and image classification all show the three regions. The power-law exponents are steeper than theory's -0.5 would suggest in some domains and shallower in others, so they must be measured.

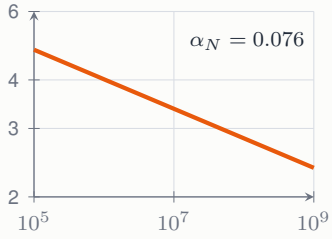
Rosenfeld et al. (2019). A joint functional form in data size and model size, fit on small configurations, extrapolated to larger ones: the first explicit "predict the big run" recipe for deep networks.

Amodei and Hernandez (2018). Compute in the largest training runs had doubled about every 3.4 months since 2012; what that compute buys had become the question.

Kaplan et al. 2020: three laws for language models

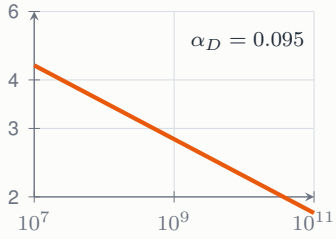
Loss is a power law in each of parameters, data and compute when the other two are not the bottleneck.

$L(N) = (N_c/N)^{\alpha_N}$



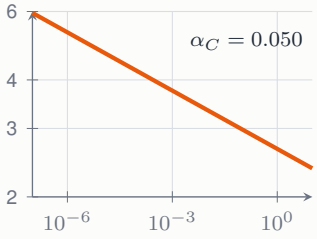
Seven decades of compute, one straight line each; $N_c = 8.8 \times 10^{13}$, $D_c = 5.4 \times 10^{13}$.

$L(D) = (D_c/D)^{\alpha_D}$



Shape barely matters. At fixed N, depth versus width moves loss by a few percent; N moves it by the law.

$L(C) = (C_c/C)^{\alpha_C}$



Joint law. $L(N, D) = \left[(N_c/N)^{\alpha_N/\alpha_D} + D_c/D\right]^{\alpha_D}$, with its own fitted constants; overfitting sets in as $N^{0.74}/D$ grows.

curves drawn from the paper's fitted constants; 1 PF-day $= 8.64 \times 10^{19}$ FLOPs

Kaplan's recipe: grow the model faster than the data

Allocating a growing budget by the 2020 fits sends most of each new decade of compute to parameters, not tokens.

what $10\times$ compute buys under each allocation rule

Kaplan 2020: $N \propto C^{0.73}$, $D \propto C^{0.27}$



Chinchilla 2022: $N \propto C^{0.5}$, $D \propto C^{0.5}$



Kaplan: $B \propto C^{0.24}$ and $S \propto C^{0.03}$, so $D = BS \propto C^{0.27}$

Two more 2020 conclusions. Larger models reach a given loss in fewer tokens, and the compute-efficient frontier stops well short of convergence: train big, stop early.

The recipe in practice: GPT-3

175B parameters on 300B tokens, about 1.7 tokens per parameter, and the largest runs of 2020–21 followed suit (Gopher: 280B on 300B). Under the Chinchilla analysis two years later, these models turned out to be under-trained for their compute.

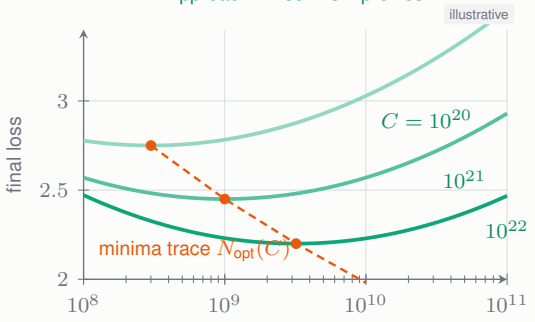
Critical batch size

Kaplan also fit $B_{\text{crit}}(L) \propto L^{-1/\alpha_B}$ with $\alpha_B \approx 0.21$: the batch below which steps and examples trade off one for one (McCandlish et al. 2018). It grows as loss falls, which is why batch size is scheduled or scaled with the run.

Chinchilla, 2022: three ways to find the compute-optimal split

Over 400 models from 70M to 16B parameters, one question: at a fixed compute budget, how big should the model be?

Approach 2: IsoFLOP profiles



Each curve: one compute budget, spent on models of different sizes (bigger N means fewer tokens). The valley bottom is the compute-optimal size.

1. **Fix N, vary D**
   Train each size on several token counts; the lower envelope of all training curves gives the best loss per FLOP and the N, D that achieve it.
2. **IsoFLOP profiles**
   Fix C, vary N (and so $D = C/6N$). Loss against $\log N$ is a valley; fit a parabola, take the minimum, repeat for nine budgets.
3. **Parametric fit**
   Fit $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ to all runs at once, then minimize it subject to $6ND = C$ analytically.

All three agree: $N_{\text{opt}} \propto C^{a}$ with $a \approx 0.5$. The agreement, not any single fit, is the evidence.

Chinchilla's result: scale parameters and tokens together

About twenty tokens per parameter at the compute-optimal point, and a 70B model that beat a 280B one at equal cost.

	Params	Tokens	Tok./param	FLOPs
Gopher (2021)	280B	300B	1.1	5.8×10^{23}
Chinchilla (2022)	70B	1.4T	20	5.8×10^{23}

Same budget, $4\times$ fewer parameters, $4.7\times$ more tokens; Chinchilla wins on nearly every evaluation and is cheaper to serve.

$$N_{\text{opt}} \propto C^{0.5}, \qquad D_{\text{opt}} \propto C^{0.5}, \qquad C \approx 6\,N\,D$$

The rule of thumb. For a budget C, choose $N \approx \sqrt{C/120}$ and $D \approx 20N$. Ten times the compute: $3.2\times$ the model and $3.2\times$ the data.

Where $6ND$ comes from

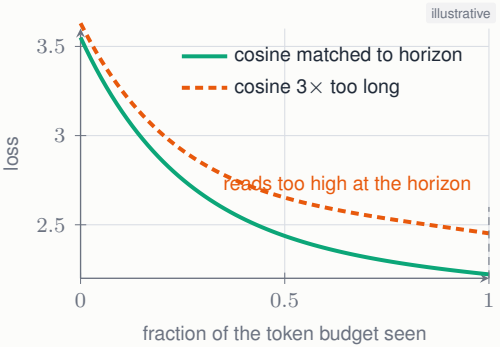
About $2N$ multiply-adds per token forward (every weight once) and $4N$ backward (gradients for activations and weights). Attention adds a term that only matters at very long context.

What the exponents mean

Approaches 1–3 gave $a = 0.50,\ 0.49,\ 0.46$ for N and $b = 0.50,\ 0.51,\ 0.54$ for D. Equal exponents: at the optimum, one parameter and twenty tokens buy the same marginal loss. The 20 is an empirical constant that moves with data quality and recipe, not a law of nature.

Why Kaplan and Chinchilla disagreed

Same question, exponents of 0.73 versus 0.5. The gap is methodology, and later papers pinned down each piece.



Kaplan read intermediate points off long runs. A schedule that has not decayed by the point you read makes short-horizon loss look worse, which favors bigger models at small budgets.

Learning-rate schedule — Hoffmann 2022
Decay must end where the loss is read; otherwise small-data points look worse than they are.

What counts as N — Porian et al. 2024
Kaplan excluded embedding parameters; at small scale the output layer is a big share of compute.

Warm-up and tuning — Porian et al. 2024
Warm-up, learning rate and batch size must be re-tuned per scale; fixed settings bias small runs.

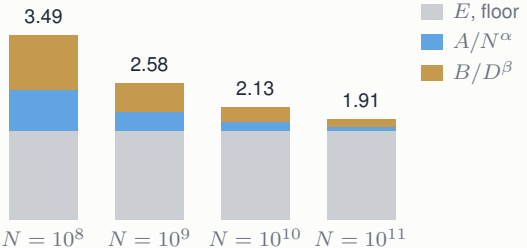
Fit hygiene — Besiroglu et al. 2024
Hoffmann's published approach-3 constants imply about 70 tokens per parameter, against 20 from approaches 1–2; a refit restores 20.

With these corrected, Kaplan-style data reproduces $a \approx 0.5$; Porian et al. find the schedule matters less than the other three. The Practice section turns this list into a checklist.

The parametric form, term by term

$L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ is the workhorse. Each term has a meaning, and the fit has a standard procedure.

loss at $D = 20N$ with Hoffmann's constants ($E = 1.69$, $A = 406.4$, $\alpha = 0.34$, $B = 410.7$, $\beta = 0.28$)



The floor dominates early; the two excess terms shrink at different rates, and their ratio at the optimum is what sets tokens per parameter.

Meaning

E: entropy of text under the ideal model. A/N^{α}: what a finite network cannot represent. B/D^{β}: what a finite sample cannot pin down.

How to fit it

Minimize a Huber loss on $\log L$ with L-BFGS from a grid of starts; keep the best. $N_{\text{opt}}(C)$ then follows by calculus, with $a = \beta/(\alpha + \beta)$. Caveat (Besiroglu et al. 2024): these published constants imply about 70 tokens per parameter; their refit gives $E = 1.82$, $\alpha = 0.35$, $\beta = 0.37$ and 20 again.

Why power laws at all?

Three pictures that produce them, each saying what the exponent depends on. None is settled; all are useful.

Resolution on a manifold

Data lies near a d-dimensional manifold. N parameters resolve it to spacing $\propto N^{-1/d}$; a smooth target then gives $L - E \propto N^{-4/d}$. Small exponent, high-dimensional data.

Sharma, Kaplan 2020; Bahri et al. 2021

Zipf-distributed features

Feature frequencies follow a power law. Each feature is learned once enough examples are seen, so the loss left after D tokens is a power law in D whose exponent is set by the Zipf slope.

Hutter 2021

Quanta of skill

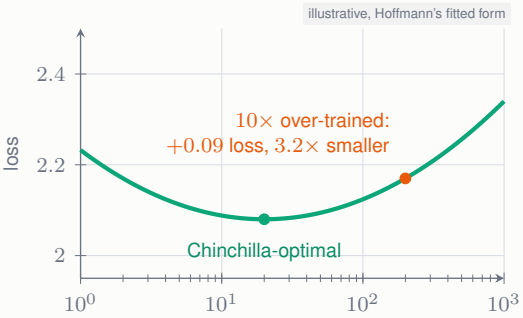
Loss is a sum over many discrete "skills", each learned at its own scale and used with Zipfian frequency. Averaged, they give a smooth power law even though each one switches on abruptly.

Michaud et al. 2023

What they agree on. The exponent belongs to the data distribution more than to the architecture: Hestness saw architectures shift curves without changing slopes, and a new data mix needs its own ladder. Bahri et al. also separate a variance-limited regime (exponent 1, when one resource dwarfs the other) from the resolution-limited one above.

Refinement I: the cost of serving changes the optimum

Chinchilla minimizes training compute. If the model will answer billions of queries, serving cost belongs in the objective.



The valley is shallow: moving ten times past the optimum costs little loss but buys a much smaller model.

Inference-aware scaling

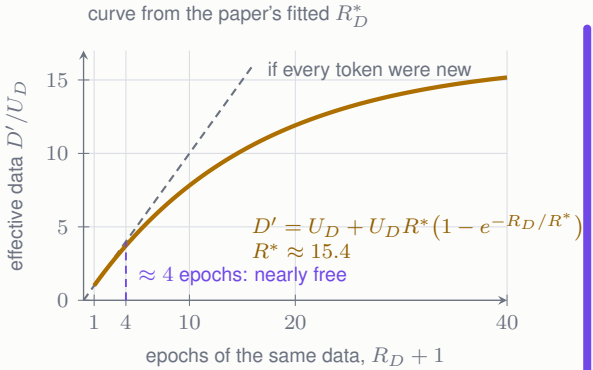
Sardana and Frankle 2023. Add expected inference FLOPs ($2N$ per generated token) to the objective. For a model that will serve enough tokens, the optimum shifts to smaller N and larger D, often far past 20 tokens per parameter.

Gadre et al. 2024. Over-training is predictable: laws fit on 104 small models forecast a run $32\times$ over-trained (1.4B parameters, 900B tokens) from $300\times$ less compute, and downstream error tracks loss through a power law.

In practice. Llama 3 8B saw 15T tokens (about 1,900 per parameter); frontier models routinely train at hundreds of tokens per parameter because serving dominates total cost.

Refinement II: when data runs out, repeat it

Unique text is finite. Muennighoff et al. measured what a repeated token is worth and fit a law with a decay constant.



U_D unique tokens repeated R_D extra times act like D' fresh tokens; the value of each new pass decays exponentially and saturates near $16\,U_D$.

What it means for a budget

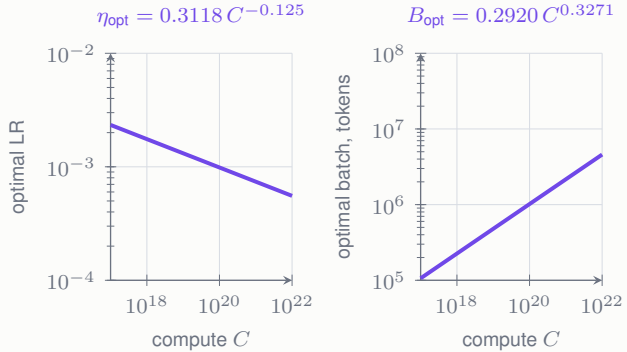
Up to about four passes over the same data, the loss is close to what unique data would give; after that each pass is worth less, and by tens of passes extra compute buys almost nothing.

Allocation changes. When data is the constraint, spend the surplus compute on more epochs and on more parameters, both with decaying returns, rather than stopping.

Why it matters now. Villalobos et al. (2022, revised 2024) project that training sets will reach the size of the public human-text stock between 2026 and 2032; frontier runs already use 15–50T tokens, so repetition is part of the design (and why MAI's ladder repeats data on every rung).

Refinement III: hyperparameters have scaling laws too

A ladder is only as good as the tuning of its rungs. Learning rate and batch size move with scale in predictable ways.



DeepSeek LLM (2024) fits from a grid of small runs. The loss basin is wide: values within a factor of two of the fit are near-optimal.

Three ways to keep rungs tuned

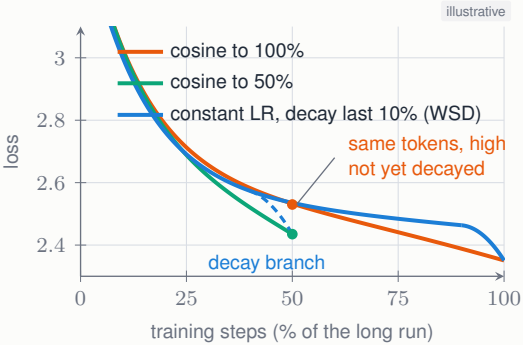
Sweep and fit. Grid-search LR and batch at a few small scales, fit power laws in C (DeepSeek LLM 2024) or in steps and tokens per parameter (MAI-Base-2), extrapolate.

Parametrize so the optimum does not move. μP (Yang et al. 2022) rescales initialization and per-layer learning rates with width so that the best LR found on a small model transfers to a wide one.

Scale the batch, not the steps. Below the critical batch size (McCandlish et al. 2018), doubling the batch halves the steps; above it, extra batch is wasted. The critical size grows as loss falls.

Refinement IV: the schedule decides what a point is worth

Loss at the end of a cosine run is not the loss at the same token count inside a longer run. Ladders must control for it.



At the 50% mark, the long cosine run is well above the run whose schedule ended there. Reading intermediate points off a long run is the Kaplan error.

Practical consequences

One run per point, each with its schedule matched to its own horizon, is the safe default. It is expensive: k token budgets per model size means k runs.

Warmup-stable-decay (Hu et al. 2024, MiniCPM; Hägele et al. 2024): hold the learning rate constant, then decay for about the last 10% of steps. Branch a short decay off the constant run at any point and you get the loss a matched-schedule run would have reached, at a fraction of the cost. Ladders become cheap in D.

What to read. Always the loss after decay; never the raw value on the constant-LR plateau, which sits well above it.

Refinement V: laws for architecture knobs

Once the recipe is fixed, the same machinery answers narrower questions: how sparse, what vocabulary, what precision

Mixture of experts

Loss follows a joint law in dense size and expert count. The gain over a dense model at equal active parameters shrinks slowly with scale but stays large; finer experts push the frontier further.
Clark et al. 2022; Krajewski et al. 2024

Vocabulary size

Vocabulary parameters scale differently from the rest. The optimal vocabulary grows with compute, more slowly than N; most models' vocabularies are smaller than the fit recommends.

Tao et al. 2024

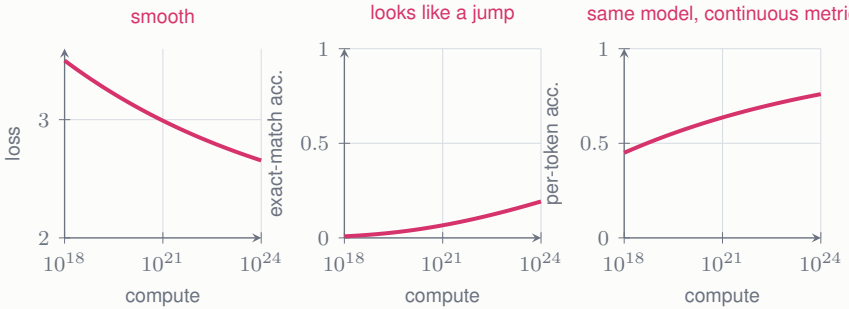
Precision

Lower precision acts like fewer effective parameters, $N_{\text{eff}} = N\, f(P)$; post-training quantization hurts more the longer a model trained. Their fit puts compute-optimal precision near 7–8 bits.
Kumar et al. 2024

Data mixtures (Ye et al. 2024): loss on each domain is a predictable function of the mixture proportions, so the mix can be chosen from small runs. **The common pattern:** fix everything else, vary one knob across a few scales, fit a law with the knob as an extra variable, and extrapolate. Each such law is only trustworthy inside the recipe it was fit on.

From loss to capability: what scaling laws do not promise

Loss is smooth. A benchmark score is a nonlinear, often thresholded, function of it, and that is where “emergence” lives.



illustrative

A six-token answer scored all-or-nothing: acc $= p^6$ turns a steady per-token gain into a late, sudden rise.

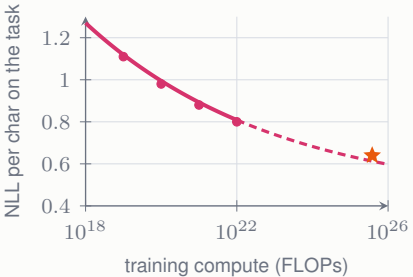
Emergent abilities (Wei et al. 2022): on many benchmarks small models score at chance and large ones suddenly do not. **The mirage view** (Schaeffer et al. 2023): the jumps track the metric, not the model; linear or probabilistic metrics on the same outputs scale smoothly.

What to take from it. Predict loss, or a continuous proxy of the task, and only then map to the discontinuous score. Both views agree that loss stays on its law; they disagree about which number you should care about.

Predicting a benchmark score: the two-stage map

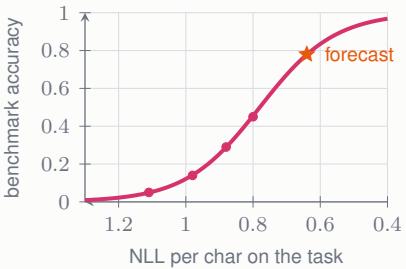
Fit compute to a continuous intermediate quantity, then that quantity to the score. Each stage is better behaved than the

stage 1: compute → loss on the task's answers



stage 2: loss → accuracy (sigmoid)

illustrative, after Llama 3



Llama 3 (2024). Scaling-law runs from 6×10^{18} to 10^{22} FLOPs predicted the 405B model's benchmark scores at 3.8×10^{25}: a $10^4\times$ extrapolation, in two stages. Stage 2 can use older, larger models as extra points, since it does not depend on compute.

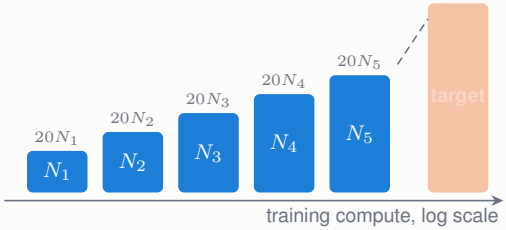
GPT-4 (2023). Final loss predicted from runs with at most $1/10{,}000$ of its compute; HumanEval pass rate from $1/1{,}000$, using the mean log pass rate as the smooth intermediate. **OLMo ladders** (Bhagia et al. 2024) do the same for open models.

The ladder: one recipe, several sizes, one fit

A scaling ladder is a set of models that differ only in scale. Everything else is held fixed so that scale is the only variable.

Held fixed: data mix, tokenizer, architecture family, optimizer, schedule shape, tokens per parameter, evaluation. Varied: N, and D with it.

five rungs, each 2–$3\times$ the compute of the last



What a rung is

One run at one scale, tuned for that scale (its own learning rate, batch and schedule), evaluated like every other rung.

What a ladder answers

Forecast: fit loss against compute, extrapolate. **Compare:** two ladders differing in one choice; the better one is lower at the target, not at the smallest rung. **Allocate:** fit $L(N, D)$ with several D per

Current examples

MAI-Base-2's ladder (L18 to L42, then L102); OLMo's model ladders (Bhagia et al. 2024).

Designing a ladder: the choices and the usual answers

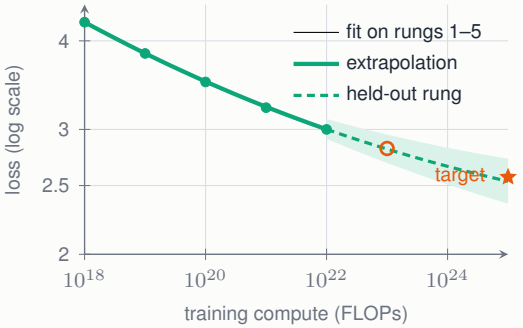
A ladder is an experiment. Decide what is fixed, what is varied, how many points, and what is measured, before any run

Choice	Usual answer	Why
How to set a rung's shape	One scalar (layers L, or width) with the rest tied to it: $d \propto L$, heads $\propto d$	keeps every rung the same "kind" of model; shape effects are small anyway (Kaplan)
How many rungs	5–8, spaced 2–$4\times$ apart in compute, spanning ≥ 2 decades	a 3-parameter fit needs more than 3 points; the span sets how far you can extrapolate
How far to extrapolate	1–2 decades past the top rung	GPT-4: 4 decades on loss worked; errors grow with distance, so keep a holdout rung
Tokens per rung	Fixed tokens per parameter (e.g. 20), or several D per N for an $L(N, D)$ fit	fixed TPP gives a one-variable curve; multiple D cost more but return the allocation
Hyperparameters	Re-tuned per rung by a law or μP; never copied from the smallest rung	an untuned rung sits above the curve and bends the fit (Porian et al.)
Schedule	Matched to each rung's horizon, or WSD with a decay branch per point	intermediate points of a long run are not valid rungs
What to measure	Held-out loss in nats or bits per byte; task NLL for the two-stage map	loss is smooth; accuracy is not
Repeats	At least two seeds on the smallest rungs	seed noise at small scale is often larger than the effect you are testing

Fitting the law and reading the forecast

Fit in log space, hold out the top rung, report the spread. A forecast without an error bar is a guess with a decimal point.

illustrative; band = spread of fits across functional forms and leave-one-out refits



Procedure

1. Choose the form: $E + A\,C^{-\alpha}$ for one variable, $E + A/N^{\alpha} + B/D^{\beta}$ when D varies. **2.** Fit in log space with a Huber loss from several starts; E is the least determined parameter, so also try fixing it. **3.** Refit leaving each rung out, and with the top rung held out; the scatter is your error bar. **4.** Check residuals: curvature means a wrong form or a mistuned rung. **5.** Convert loss gaps to compute ratios: $\Delta \log L$ at slope α is a factor $10^{\Delta \log L/\alpha}$.

Reading the band. With $\alpha \approx 0.08$, a 1% error in predicted loss is a $1.13\times$ error in equivalent compute; 5% is $1.8\times$. Loss forecasts are precise; compute-equivalence forecasts are not.

Worked example: a $1M scaling study

Fictional but realistic numbers. Convert dollars to FLOPs, choose the rungs, decide what each can and cannot tell you.

Budget to FLOPs. $1M at $2.50 per H100-hour is 400k GPU-hours. At 989 TFLOPS BF16 peak and 40% MFU, one GPU-hour is 1.4×10^{18} FLOPs, so the budget is about 5.7×10^{23} FLOPs (derived).

Rung	N	$D = 20N$	FLOPs	GPU-h
1	50M	1B	3×10^{17}	0.2
2	150M	3B	2.7×10^{18}	2
3	400M	8B	1.9×10^{19}	14
4	1B	20B	1.2×10^{20}	85
5	3B	60B	1.1×10^{21}	760
holdout	8B	160B	7.7×10^{21}	5,400

Rungs 1–5 for three variants and two seeds: about 5,200 GPU-hours. With the holdout, under 3% of the budget; a 30B target run at 10^{23} FLOPs would be 76k GPU-hours, 19%.

What this ladder can tell you

Forecast the loss of a 30B/600B-token run at 10^{23} FLOPs, two decades past rung 5, with the 8B holdout as the check.

Compare three recipe variants (say, two data mixes and a tokenizer) as compute multipliers at the target, if their curves have separated by rung 5.

Sanity checks before trusting it

Rungs 1–2 are seed-noise dominated: plot both seeds. Rung 5 alone is larger than the rest combined: if it fails, the ladder is four points. Keep the evaluation set fixed and decontaminated from the first run to the last.

Pitfalls: how scaling experiments go wrong

Most failed forecasts trace back to one of these. Each has shown up in a published paper at least once.

1 Points off one long run

Intermediate losses of a cosine run overstate what a matched-horizon run would reach. Use one run per point or a WSD branch.

2 Copying hyperparameters up

The best LR shrinks and the best batch grows with scale. An untuned rung sits above the curve and tilts the fit.

3 Extrapolating too far

Two decades past the top rung is routine; four worked for GPT-4 on loss. Keep a holdout rung and report the spread of refits.

4 Comparing at the wrong scale

Recipe A beats B at 50M and loses at 3B. Compare curves at the target, and only call it when the curves have separated beyond seed noise.

5 Changing two things at once

A new tokenizer changes the loss unit (nats per token); compare in bits per byte, and change one variable per ladder.

6 Trusting the floor

E is the least determined constant and dominates long extrapolations. Fit with and without it; if forecasts diverge, you need a higher rung.

Also: evaluation sets contaminated by training data (the floor looks lower than it is); loss measured on a different distribution than the one the law was fit on; embedding parameters counted inconsistently across rungs; and mixing runs with different precision or numerics.

The frontier: where the laws are being pushed

The 2020–22 picture still holds for dense pretraining. Four developments change what is on the axes.

Sparse models

Mixture-of-experts models separate parameters from compute per token. Laws now carry two sizes (total and active) and an expert count; MAI-Base-2 at 2.8T/80B is a current data point.

Data as the constraint

Public text is finite. Repetition laws (Muennighoff), synthetic data and curation become part of the recipe, and ladders must repeat data on every rung to stay honest.

Laws for the whole recipe

Learning rate, weight decay, dropout, vocabulary, precision and mixture each get a fitted law. The ladder is now a system that predicts the full configuration of the big run, not only its loss.

Beyond the pretraining loss

The loss floor and the two-stage map give a benchmark forecast. What they do not cover is post-training or compute spent at inference, which change capability without touching the pretraining law.

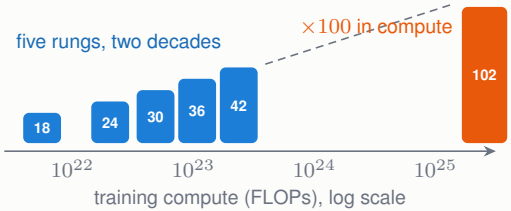
The constant through all of it: a straight line on log axes, measured on a ladder, extrapolated with an error bar. The axes change; the method does not.

Case study: the MAI-Base-2 ladder (2026)

Everything in this lecture, applied once: five rungs over two decades, a forecast two decades out, and the forecast held.

Rungs L18 to L42 with width $= \frac{256}{3}L$, heads $= L$, fixed tokens per active parameter, and each rung repeating its data as often as the full run will (source scaling).

Final model: 2.8T total, 80B active, 50T tokens, about 2.4×10^{25} FLOPs on 16k GB200 GPUs over 38 days.



Source: Microsoft AI, Frontier Pre-training at Microsoft AI (Oct 2026); Part Two of this series covers it in full.

What the ladder produced

A forecast of the final evaluation that landed on the measured value after extrapolating over $100\times$ in compute.

Compute-equivalence claims: external models projected onto the curve give 4–$12\times$ FLOPs-efficiency over the strongest open base at equal bits per byte.

Hyperparameter laws for learning rate, weight decay and dropout, fit small, validated mid-scale.

Scale-dependence caught: one change gained $+2\%$ at L18 and $+28\%$ at L42.

Exercises

Three short ones to do by hand, two that need a GPU. Answers to the first three are on the next slide.

1. **Allocate.** You have 10^{22} FLOPs. Using Chinchilla's rule, what N and D should you train? What if you instead expect to serve 10^{14} tokens over the model's life, so that inference compute $2N \times 10^{14}$ counts too?

2. **Convert.** Two recipes differ by 0.02 nats at the top rung and the ladder's slope is $\alpha = 0.08$ in $L = E + A\,C^{-\alpha}$ with $L - E \approx 0.9$ there. Express the difference as a compute multiplier. Why does the answer depend on E?

3. **Repeat.** With $U_D = 5$T unique tokens and Muennighoff's $R_D^* = 15.4$, how many effective tokens do 8 passes give? What fraction of 40T fresh tokens is that?

4. **Build (GPU).** Train a five-rung ladder of small transformers (1M–50M parameters, 20 tokens per parameter) on one corpus with WSD schedules. Fit $E + A\,C^{-\alpha}$, hold out the top rung, report the forecast and its leave-one-out spread.

5. **Break it (GPU).** Repeat exercise 4 reading intermediate points off one long cosine run instead. Compare the fitted exponent and the holdout error with those of the matched-schedule ladder.

Worked answers to exercises 1–3

Showing the arithmetic, since the point of the lecture is that these are back-of-envelope calculations.

1. Allocate

Chinchilla: $N = \sqrt{C/120} = \sqrt{10^{22}/120} \approx 9.1$B, $D = 20N \approx 180$B tokens.

With serving, total cost is $6ND + 2N \cdot 10^{14}$. A 3B model trained to the same loss costs more to train but saves about 1.2×10^{24} serving FLOPs, more than the training budget: choose it.

2. Convert

Only the excess over the floor scales: $\log_{10}(0.92/0.90) \approx 0.0096$; at slope $\alpha = 0.08$ the multiplier is $10^{0.0096/0.08} \approx 1.32$.

Using the full loss $L \approx 2.6$ instead gives $\log_{10}(2.62/2.60)/0.08 \approx 0.042$ and only $1.10\times$. The floor decides how much of the loss is in play, so an uncertain E makes compute-equivalence claims uncertain even when the loss forecast is tight.

3. Repeat

$R_D = 7$ extra passes: $D' = 5\text{T}\left[1+15.4\,(1-e^{-7/15.4})\right] \approx 5\text{T}\times 6.6 \approx 33\text{T}$, which is 82% of 40T fresh tokens. Eight passes lose less than a fifth of their nominal value.

These use the lecture's illustrative constants; with a real ladder, substitute your own fit and carry its error bar through each step.

Summary: what to remember

Six statements that survive every paper in this lecture.

Loss follows power laws

in N, D and C, over many decades, with a floor. The exponent belongs to the data more than to the architecture.

Balance N and D

Compute-optimal training puts roughly twenty tokens on each parameter; serving cost pushes you to smaller models trained longer.

Everything scales

Learning rate, batch, weight decay, vocabulary, precision and the value of repeated data all have their own laws.

A ladder is the instrument

One recipe, several sizes, each rung tuned and scheduled for itself; fit in log space; hold out the top rung.

Loss first, tasks second

Benchmarks jump because metrics threshold; a continuous intermediate makes the forecast possible.

Report the error bar

Loss forecasts are precise; compute-equivalence claims amplify small errors by $1/\alpha$. Say which one you are making.

If you read three papers: Kaplan et al. 2020, Hoffmann et al. 2022, and Porian et al. 2024 (how to reconcile the first two). Then Muennighoff et al. 2023 for data, and the Llama 3 report for a forecast that was tested.

Sources

arXiv identifiers, fitted constants and quoted numbers were checked against the papers.

Before language models

Cortes et al. 1993. Learning Curves: Asymptotic Values and Rate of Convergence. NIPS 6
Banko, Brill 2001. Scaling to Very Very Large Corpora for Natural Language Disambiguation. ACL
Halevy, Norvig, Pereira 2009. The Unreasonable Effectiveness of Data. IEEE Intell. Syst. 24(2)
Sun et al. 2017. Revisiting Unreasonable Effectiveness of Data in Deep Learning Era. arXiv:1707.02968
Hestness et al. 2017. Deep Learning Scaling is Predictable, Empirically. arXiv:1712.00409
Amodei, Hernandez 2018. AI and Compute. OpenAI blog
Rosenfeld et al. 2019. A Constructive Prediction of the Generalization Error Across Scales. arXiv:1909.12673

The core results

Kaplan et al. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361
Hoffmann et al. 2022. Training Compute-Optimal Large Language Models. arXiv:2203.15556
Besiroglu et al. 2024. Chinchilla Scaling: A replication attempt. arXiv:2404.10102
Porian et al. 2024. Resolving Discrepancies in Compute-Optimal Scaling. arXiv:2406.19146
McCandlish et al. 2018. An Empirical Model of Large-Batch Training. arXiv:1812.06162

Refinements

Sardana, Frankle 2023. Beyond Chinchilla-Optimal: Accounting for Inference. arXiv:2401.00448
Gadre et al. 2024. Language models scale reliably with over-training. arXiv:2403.08540
Muennighoff et al. 2023. Scaling Data-Constrained Language Models. arXiv:2305.16264
Villalobos et al. 2022. Will we run out of data? arXiv:2211.04325
DeepSeek-AI 2024. DeepSeek LLM. arXiv:2401.02954
Yang et al. 2022. Tensor Programs V (μTransfer). arXiv:2203.03466
Hu et al. 2024. MiniCPM. arXiv:2404.06395
Hägele et al. 2024. Scaling Laws Beyond Fixed Training Durations. arXiv:2405.18392

Laws for architecture knobs

Clark et al. 2022. Unified Scaling Laws for Routed Language Models. arXiv:2202.01169
Krajewski et al. 2024. Scaling Laws for Fine-Grained Mixture of Experts. arXiv:2402.07871
Tao et al. 2024. Scaling Laws with Vocabulary. arXiv:2407.13623
Kumar et al. 2024. Scaling Laws for Precision. arXiv:2411.04330
Ye et al. 2024. Data Mixing Laws. arXiv:2403.16952

Loss to capability

Wei et al. 2022. Emergent Abilities of Large Language Models. arXiv:2206.07682
Schaeffer, Miranda, Koyejo 2023. Are Emergent Abilities a Mirage? arXiv:2304.15004
OpenAI 2023. GPT-4 Technical Report, Section 3. arXiv:2303.08774
Llama Team 2024. The Llama 3 Herd of Models, Section 3.2.1. arXiv:2407.21783
Bhagia et al. 2024. Task Scaling Laws via Compute-Efficient Model Ladders. arXiv:2412.04403

Why power laws

Bahri et al. 2021. Explaining Neural Scaling Laws. arXiv:2102.06701
Hutter 2021. Learning Curve Theory. arXiv:2102.04074
Michaud et al. 2023. The Quantization Model of Neural Scaling. arXiv:2303.13506

Case study

Microsoft AI 2026. Frontier Pre-training at Microsoft AI. microsoft.ai/news

About the figures

Curves labelled with a paper use its published constants; “illustrative” curves are sketches, not data. The $1M example is fictional.